



TPF
Technologieplattform

Fabrikstrasse 8
Büro A209
CH-3012 Bern

+41 31 6845513

v.i.b.r.i.o.n
The manual - V2.0

Carlo Prelz - carlo.prelz@unibe.ch
22nd July 2025

Contents

1	Introduction	3
1.1	The devices	3
1.2	The application	4
1.3	Data production	5
2	Practice	7
2.1	Required hardware	7
2.2	Installation	7
2.2.1	Packages	8
2.2.2	System variable	8
2.2.3	Direct connection	8
2.2.4	Installation from a <i>bare repository</i>	11
2.3	Running v.i.b.r.i.o.n	13
2.3.1	XML configuration file	13
2.3.2	Application run	18
2.3.3	Stopwatch	21
2.3.4	Metronome	22
2.3.5	Graphs	22
2.3.6	Free-form markers	24
2.3.7	Data production	24
	Appendices	26
A	Configuration variables	27
B	Audio setup	29

Chapter 1

Introduction

1.1 The devices

The software described in this document is used to connect via *Bluetooth* a Linux PC to devices produced by Irish company Shimmer¹ that contain motion sensors, to configure them as required, and to stream live data from them.

Specifically, the software targets so-called *Series 3 IMU units*². These are units that contain the three sensors that permit *inertial measurement* - that is, the evaluation of linear and rotational motion in space:

- A three-axis *accelerometer*, measuring the varying effect on the object of gravity during quiet and motion.
- A three-axis *gyroscope*, measuring the instantaneous values of rotating motion that the device is subject to.
- A three-axis *magnetometer*, measuring the current effect of the earth's magnetic field on the device.

Actually, two accelerometers are present on-board. The first one is defined as *low-noise*; the second as *wide-range*.

Each device has an on-board *Bluetooth* radio. This solution allows several devices to be active and generating live data at the same time.

The devices are charged and configured via a *USB*-connected base station.

This software only focuses on the Bluetooth connectivity part. Connection of the USB cable to the computer running *v.i.b.r.i.o.n* is *not* required. Users are supposed to make use of the software distributed by the hardware producers to make sure the proper firmware is loaded in each device.

This software has been developed by reverse-engineering the freely-distributable *c-sharp* code base distributed by the producers, available at the following address:

<https://github.com/ShimmerEngineering/Shimmer-C-API>

The above code is, sadly, totally unencumbered by any form of organic documentation. I am thus glad to state that, as requested by the copyright note included in that code,

the material described by this document is carefully abstaining from using either the name of Shimmer Research, Ltd. or the names of the code's contributors to endorse or promote this product.

Reverse-engineering work has been particularly difficult because the code mixes material for various device versions and models, and one can perceive the absence of any effort by the producers to operate the needed refactoring when they moved from version 2 to version 3 of the hardware, as well as when facing requirements imposed by models with new functionalities.

¹<https://shimmersensing.com/>

²<https://shimmersensing.com/product/shimmer3-imu-unit/>

I also want to add that the protocol implemented by the devices, although being stream-oriented (piggy-backing serial-based communication over the connection provided by the Bluetooth subsystem), does not make a clear effort at solid packetization, nor provides a clean separation between the message-oriented conversation that takes place when configuring a device and the data-packet stream that is started when configuration is completed. Furthermore, no effort seems to have been made to provide any form of recovery from data stream glitches.

On the other hand, I have been relieved to find out that no form of encryption or effort to obfuscate the protocol are in place. The absence of any protocol documentation is probably sufficient to dissuade anyone but the most stubborn to develop alternative code to converse with Shimmer devices.

Due to the lack of documentation available, especially to the absence of an organic document with details about each protocol message, I cannot be sure of the exactness of the protocol I reconstructed. Streaming takes place, and incoming data have been mapped to results that are roughly correspondent to the motion of the devices.

An effort has been made to try and map the outputs of the four main sensors (integer values of various resolutions and formats) to the output generated by the Shimmer data collection software, that modifies signs and the order of axes, and attributes units of measurement to the produced results. On the other hand, no effort could be made to verify the precision of the data being produced. *Specifically, while messages appear to be present in the protocol to submit the sensors to some sort of calibration, no effort was made to intuit how the calibration procedure should be performed.* Furthermore, while configuration values have been extracted from freshly-configured devices, and these values are fed to each device every time it is connected, *I have not been able to associate precise meanings to those messages.*

This software is only suited to basic version 3 IMU devices. These devices must be equipped with so-called **Log-and-stream** firmware.

1.2 The application

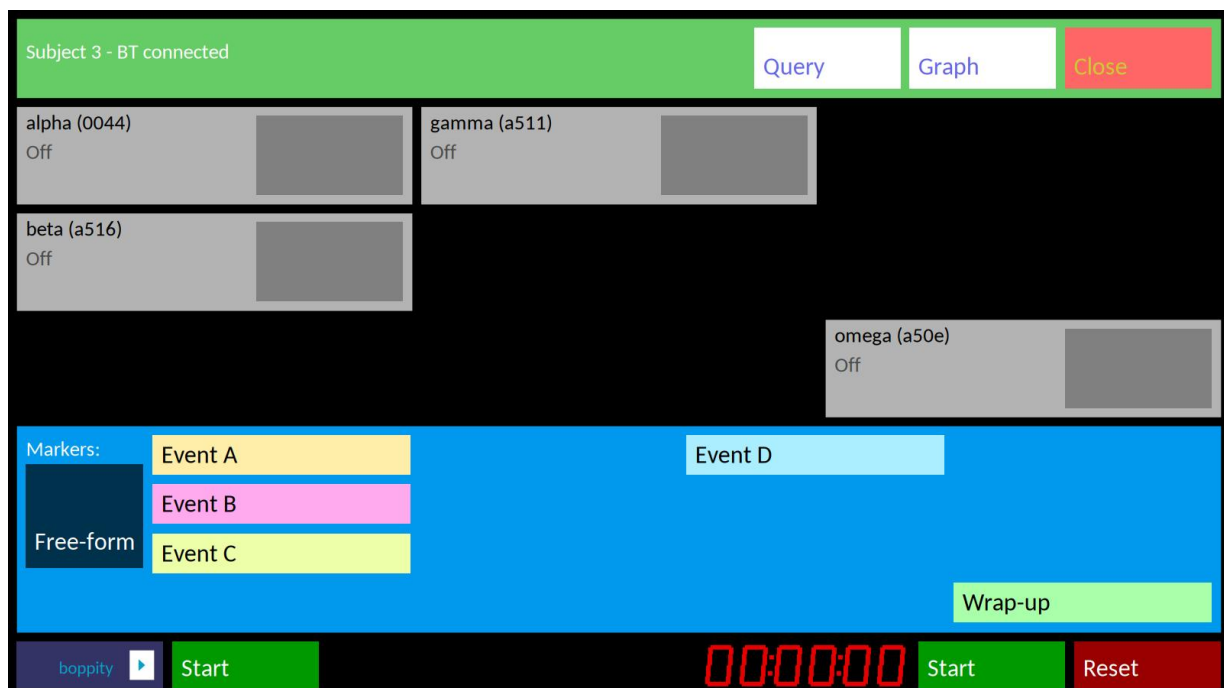


Figure 1.1: Main screen at startup

The goals of the application presented in this document are as follows:

- establish a link to as many of the devices as possible.

The characteristics of the Bluetooth wireless protocol impose a limit on the maximum number of devices that can be contacted at a time.

Experimenting with a few hardware models, I detected for each of them a limit of **7** devices connected at the same time. This seems to be dictated by the configuration of the **USB** bus that provides data exchange between the server and the Bluetooth hardware.

If contemporary connection to more than 7 Shimmer devices were to be needed, it *might* be possible to make use of more than one Bluetooth device on the server. I have no idea whether the two could easily share the same radio wave band.

In the current code, you can register more than 7 devices, and they will be correctly detected. At the moment of making connection with them, though, only the first 7 devices will succeed.

- Configure each device according to the specific needs of the application.

As already mentioned, the hardware providers offer no reference manual for the various configurable items. In the current version of the software, I limit myself to offering a series of parameter values that I desumed by reverse-engineering, which, I believe, configure these aspects:

- Calibration
- Sensor ranges
- Capture rate

I collected the current values from devices configured with Shimmer's software and they are currently hard-coded, in order to make sure comparable data are produced.

The only element that is configurable by the user is which subset of the on-board sensors will be active and generating data for each device.

- Start data streaming for those devices that are found, and save all raw streamed material on-disk, together with time reference, in order to permit synchronization among the data produced by the different devices.
- Permit monitoring of link states and received data, and re-establishment of links that were eventually interrupted.
- During sessions, permit the triggering of various time markers, that will have no consequence on the operativity of the software, but will be recorded with their trigger time along with data from the devices.
- A second-resolution stopwatch is included in the user interface. It can be started, stopped and reset.
- A sound generator is included, for production of cyclical patterns (metronome function).

1.3 Data production

The main application saves binary files with raw data in the format that is received by the devices.

A utility is provided to convert captured data into comma-separated-value files that may be imported into spreadsheet programs for post-processing.

Data from the four sensors included in each device undergo manipulation to make them match data that is generated by the Shimmer application:

- For all sensors, the values of the **X** and **Y** axes are inverted.
- The sign for all values is inverted, except for:
 - values for the received axis **Y** (that becomes axis **X**) of the wide-range accelerometer.
 - values for the received axis **Y** (that becomes axis **X**) of the magnetometer.
- These factors are applied to received data, once their integer values are mapped to range 0.0...1.0:

`Low-noise accelerometer` 44.510872
`wide-range accelerometer` 19.609216
`gyroscope` 500.259542
`magnetometer` 49.125937

- Values produced from the `Low-noise accelerometer` are furthermore submitted to an offset of 24.489133.

All the constants mentioned above have been obtained from the comparison of raw collected data and the final data produced by the Shimmer application. The resulting file labels the values with the following units of measurement:

`Low-noise accelerometer` m/(s²)
`wide-range accelerometer` m/(s²)
`gyroscope` deg/s
`magnetometer` local_flux (whatever is meant by that)

Examples about how to run this application will be provided in the second part of this manual.

Chapter 2

Practice

2.1 Required hardware

v.i.b.r.i.o.n is written in **C** and **Ruby**. The required operating system is **Linux** (a derivative of **Debian** is suggested).

While the Shimmer package opts to make use of shell-level utilities to establish the Bluetooth link to the various devices, limiting itself to dealing with serial-port-like device files, **v.i.b.r.i.o.n** makes use of the underlying Bluetooth hardware by calling functions included in the library interface offered by the Linux protocol stack (see <https://www.bluez.org>). The library is accessible from Debian derivatives by loading package **libbluetooth-dev**.

I have limited experience of the various chipsets that are currently available, but the few I tested have worked painlessly.

To verify that you have a working Bluetooth environment, you execute this command:

```
hcitool dev
```

which is expected to reply with something like:

```
Devices:
hci1      88:D8:2E:43:68:DF
hci0      00:09:DD:10:6B:87
```

The above reply comes from a machine that is equipped with two devices. This is certainly uncommon: most Bluetooth usage cases manage all that is needed via a single device.

As mentioned above, more than one device may be employed to connect with more than 7 Shimmer devices, but this usage case has still not been explored. As of now, if you have more than one bluetooth device, make sure that you choose the one with better characteristics.

In most of the cases, the command will return only one output, corresponding to device **hci0**. This is what is needed to run **v.i.b.r.i.o.n**. **If the command returns NO devices, either your server has no Bluetooth device onboard, or whatever device is present is not properly supported by your Linux installation.** In both cases, you cannot make use of this application.

2.2 Installation

The installation can proceed in two ways. Either you have access to the internal network of the University of Bern and you have an account with the **TPF** Gitlab repository, or you have received a so-called *bare repository file*.

2.2.1 Packages

Here I propose a list of Debian packages that your installation may need to have loaded before you proceed to install **v.i.b.r.i.o.n**. *The list is derived from the **Placo** installation documentation, and certainly includes packages that are not needed here.* Since disk space is rarely a problem nowadays, I do not judge it necessary to prepare a specific list for this application.

```
sudo apt-get install openssh-server mc vim ruby ruby-dev emacs gpm git gcc make flex bison bc zathura
↳ xinput libusb-1.0-0-dev cmake isc-dhcp-server jwm tcpdump dnsutils wireshark libssl-dev
↳ libreadline-dev libffi-dev libelf-dev libglvnd-dev zlib1g-dev libglib2.0-dev libgtk-3-dev
↳ libxml2-dev libglew-dev libx11-xcb-dev libxcb-ewmh-dev libhidapi-dev libasound-dev
↳ libavformat-dev libswscale-dev libcomedi-dev libgirepository1.0-dev libmagickcore-dev libyaml-dev
↳ lshw gitk ntpdate libgsl-dev cmake-curses-gui iptables apparmor-utils rsync gdb ffmpeg
↳ libbluetooth-dev fonts-crosextra-carlito fonts-noto-mono fonts-crosextra-carlito
```

Once you have a working Ruby interpreter, it is also necessary to load on your system a few Ruby *gems*. Here follows the necessary command:

```
sudo gem install -V pkg-config xmlrpc git cairo pango
```

A further step is necessary, and it must be repeated whenever you upgrade your Ruby interpreter. The executable must be assigned special authorization in order to be allowed to execute configuration steps of the bluetooth subsystem. This is obtained by executing the following step:

```
sudo setcap cap_net_raw,cap_net_admin+eip $(eval readlink -f `which ruby`)
```

2.2.2 System variable

A system variable has to be defined in order for the Ruby interpreter to find the required libraries. You should either run on your terminal, or include in a script that is executed at login (e.g. **.bashrc** in your home directory), the following command:

```
export RUBYLIB=./libs
```

2.2.3 Direct connection

Make sure that the machine is online within the University network. You should be able to see the gitlab machine. Run:

```
ping phhum-a209-cp.unibe.ch
```

and you should see an output similar to this one:

```
PING phhum-a209-cp.unibe.ch (130.92.158.16) 56(84) bytes of data.
64 bytes from phhum-a209-cp.unibe.ch (130.92.158.16): icmp_seq=1 ttl=64 time=0.276 ms
64 bytes from phhum-a209-cp.unibe.ch (130.92.158.16): icmp_seq=2 ttl=64 time=0.466 ms
64 bytes from phhum-a209-cp.unibe.ch (130.92.158.16): icmp_seq=3 ttl=64 time=0.432 ms
...
...
```


Also make sure that you either know your gitlab user and password, or you have uploaded your **ssh** public key to the gitlab server.

Position yourself in the directory where you want to install **v.i.b.r.i.o.n**. Then execute this command:

```
git clone gh@phhum-a209-cp.unibe.ch:RBY/RBY-Shimmer2.git
```

Type your user and password if required. Then you'll see an output similar to this one:

```
Cloning into 'RBY-Shimmer2'...
remote: Counting objects: 239, done.
remote: Compressing objects: 100% (120/120), done.
remote: Total 239 (delta 109), reused 158 (delta 68)
Receiving objects: 100% (239/239), 674.94 KiB | 61.36 MiB/s, done.
Resolving deltas: 100% (109/109), done.
```

and you will find that directory **RBY-Shimmer2** has been created for you.

Change into that directory, and run:

```
ruby scripts/installer.rb
```

You will receive an output similar to what follows:

```
[240812.110509] Executing <cd /tmp/z/RBY-Shimmer2/ && git submodule update --init --recursive>
Submodule 'sub/cutlils' (gh@phhum-a209-cp.unibe.ch:RBY/RBY-C-utlils.git) registered for path
↳ 'sub/cutlils'
Cloning into '/tmp/z/RBY-Shimmer2/sub/cutlils'...
Warning: No xauth data; using fake authentication data for X11 forwarding.
X11 forwarding request failed on channel 0
Submodule path 'sub/cutlils': checked out '5181e3f602c833399813691f3908963d2cd40f3d'
[240812.110510] Executing <cd /tmp/z/RBY-Shimmer2// && git checkout master>
Already on 'master'
Your branch is up to date with 'origin/master'.
[240812.110510] Executing <cd /tmp/z/RBY-Shimmer2//sub/cutlils/ && git checkout master>
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/carlobase /tmp/z/RBY-Shimmer2/ext/carlobase
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/maths /tmp/z/RBY-Shimmer2/ext/maths
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/gr /tmp/z/RBY-Shimmer2/ext/gr
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/cmidi /tmp/z/RBY-Shimmer2/ext/cmidi
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/cairoplot /tmp/z/RBY-Shimmer2/ext/cairoplot
ln -sf /tmp/z/RBY-Shimmer2/sub/cutlils/ext/caudio /tmp/z/RBY-Shimmer2/ext/caudio
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
ln -sf /tmp/z/RBY-Shimmer2/ext/shimmer sub/cutlils//ext/shimmer
[240812.110510] Executing <(cd ext/shimmer && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
compiling shimmer2_bt.c
compiling shimmer2_gr.c
compiling shimmer2_ui.c
linking shared-object shimmer2.so
[240812.110511] Executing <(cd sub/cutlils/ext/carlobase && ruby extconf.rb && make clean && make
↳ -j12)>
```

```

creating Makefile
compiling arena.c
compiling arturia.c
compiling chronoidea.c
compiling init.c
compiling lili2.c
compiling logthread.c
compiling md5.c
compiling time_avg.c
compiling xml.c
linking shared-object carlobase.so
[240812.110511] Executing <(cd sub/cutils/ext/maths && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling accsin.c
compiling averager.c
compiling casteljau.c
compiling catmull.c
compiling clusterer.c
compiling damped.c
compiling init.c
compiling smoother.c
compiling varmat.c
linking shared-object maths.so
[240812.110513] Executing <(cd sub/cutils/ext/gr && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling btnw.c
compiling init.c
compiling sevseg.c
linking shared-object gr.so
[240812.110514] Executing <(cd sub/cutils/ext/cmidi && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
linking shared-object cmidi.so
[240812.110514] Executing <(cd sub/cutils/ext/cairoplot && ruby extconf.rb && make clean && make
↳ -j12)>
creating Makefile
compiling cairoplot.c
compiling init.c
linking shared-object cairoplot.so
[240812.110514] Executing <(cd sub/cutils/ext/caudio && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
compiling metron.c
linking shared-object caudio.so
[240812.110515] Compiling ext/shimmer
[240812.110515] Executing <(cd ext/shimmer && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
compiling shimmer2_bt.c
compiling shimmer2_gr.c
compiling shimmer2_ui.c
linking shared-object shimmer2.so
[240812.110515] Compiling sub/cutils/ext/carlobase
[240812.110515] Executing <(cd sub/cutils/ext/carlobase && ruby extconf.rb && make clean && make
↳ -j12)>
creating Makefile
compiling arena.c
compiling arturia.c
compiling chronoidea.c
compiling init.c
compiling lili2.c
compiling logthread.c
compiling md5.c
compiling time_avg.c

```

```

compiling xml.c
linking shared-object carlobase.so
[240812.110516] Compiling sub/cutils/ext/maths
[240812.110516] Executing <(cd sub/cutils/ext/maths && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling accsin.c
compiling averager.c
compiling casteljau.c
compiling catmull.c
compiling clusterer.c
compiling damped.c
compiling init.c
compiling smoother.c
compiling varmat.c
linking shared-object maths.so
[240812.110517] Compiling sub/cutils/ext/gr
[240812.110517] Executing <(cd sub/cutils/ext/gr && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling btnw.c
compiling init.c
compiling sevseg.c
linking shared-object gr.so
[240812.110518] Compiling sub/cutils/ext/cmidi
[240812.110518] Executing <(cd sub/cutils/ext/cmidi && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
linking shared-object cmidi.so
[240812.110518] Compiling sub/cutils/ext/cairoplot
[240812.110518] Executing <(cd sub/cutils/ext/cairoplot && ruby extconf.rb && make clean && make
↵ -j12)>
creating Makefile
compiling cairoplot.c
compiling init.c
linking shared-object cairoplot.so
[240812.110519] Compiling sub/cutils/ext/caudio
[240812.110519] Executing <(cd sub/cutils/ext/caudio && ruby extconf.rb && make clean && make -j12)>
creating Makefile
compiling init.c
compiling metron.c
linking shared-object caudio.so
[240812.110519] Installing ext/shimmer
[240812.110519] Installing sub/cutils/ext/carlobase
[240812.110519] Installing sub/cutils/ext/maths
[240812.110519] Installing sub/cutils/ext/gr
[240812.110519] Installing sub/cutils/ext/cmidi
[240812.110519] Installing sub/cutils/ext/cairoplot
[240812.110519] Installing sub/cutils/ext/caudio

```

Installation of **v.i.b.r.i.o.n** will now be complete.

2.2.4 Installation from a *bare repository*

You will have received a file named `shimmer2_bare_repos.YYMMDD.hhmmss.tar.xz`.¹ Copy it wherever you want to install **v.i.b.r.i.o.n**. Then extract it with:

```
tar xvf shimmer2_bare_repos.YYMMDD.hhmmss.tar.xz
```

¹This file is generated from a working installation by running a script found in the repository. The script is called `scripts/barer.rb`, and when run it will dump the produced bare repository file in the `/tmp` directory.

A new directory called **git** will be created. At this point you can discard the bare repository file.

Then run:

```
git clone git/g_generic/shimmer.git
```

Directory **shimmer** will now have been created. Change to it, and run:

```
ruby scripts/installer.rb ../git/
```

In case you receive an error: similar to this

```
fatal: transport 'file' not allowed
```

you should make sure that your **GIT** installation permits the usage of file-based repositories. Please run this command:

```
git config --global protocol.file.allow always
```

and then start the process from the beginning.

Note that the command is *almost* identical to the one that is run to install when connected to the University network. Two dots and a slash are appended to the command: they indicate to the script that it is to look for git repositories in a directory on the local machine - specifically, the parent directory of the one where you are currently located - instead of on the net.

The output of this command is analogous to the one shown in the previous section, and if all runs without errors, you will have a working installation equivalent to the one obtained above.

The main functional difference between the two modes of installation is that with the direct connection method you can directly update to newer code (using command

```
git pull
```

and then executing:

```
ruby scripts/update.rb -A
```

while with the bare repository approach it is advisable for you to proceed with a completely new installation when you receive a new bare repository file.

2.3 Running v.i.b.r.i.o.n

2.3.1 XML configuration file

```
<vibrion data_dir='/tmp' bt_unit='0'>
  <dev id='0044' tag='alpha'>
    <screen_pos x='0' y='0' />
    <graph_pos x='0' y='0' />
    <data_coll elems='D_ACCEL,LSM303DLHC_MAG,MPU9150_GYRO' />
  </dev>
  <dev id='a516' tag='beta'>
    <screen_pos x='0' y='1' />
    <data_coll elems='A_ACCEL,D_ACCEL,LSM303DLHC_MAG,MPU9150_GYRO' />
  </dev>
  <dev id='a511' tag='gamma'>
    <screen_pos x='1' y='0' />
    <data_coll elems='A_ACCEL,D_ACCEL,LSM303DLHC_MAG,MPU9150_GYRO' />
  </dev>
  <dev id='a50e' tag='omega'>
    <screen_pos x='2' y='2' />
    <graph_pos x='1' y='1' />
    <data_coll elems='A_ACCEL,LSM303DLHC_MAG,MPU9150_GYRO' />
  </dev>
  <marker x='0' y='0' col='ffeeaa' tag='Event A' type='two_btn' />
  <marker x='0' y='1' col='ffaaee' tag='Event B' type='one_btn' />
  <marker x='0' y='2' col='eeffaa' tag='Event C' type='one_btn' />
  <marker x='2' y='0' col='aaeeff' tag='Event D' type='two_btn' />
  <marker x='3' y='3' col='aaffaa' tag='Wrap-up' type='simple' />
  <metron card='sofhdadsp' dev='0' samplerate='16000'>
    <pattern name='boppity'>
      <bip freq='440.0' len='0.1' vol='0.9' />
      <silence len='0.4' />
      <bip freq='523.551' len='0.1' vol='0.6' />
      <silence len='0.15' />
      <bip freq='523.551' len='0.1' vol='0.6' />
      <silence len='0.15' />
    </pattern>
    <pattern name='bep'>
      <bip freq='1046.5' len='0.25' vol='0.8' />
      <silence len='0.25' />
    </pattern>
    <beat name='bup' bfreq='1.25' on='0.02' afreq='2093.0' vol='0.5' />
  </metron>
  <windows uiw='1600' uih='900' gw='1280' gh='900' />
</vibrion>
```

Figure 2.1: Example configuration file

The configuration file permits you to:

- Specify where you want the produced raw material for your sessions to be saved.
- Specify which Bluetooth device to use.
- Specify which Shimmer devices you will want to use, what identifier tag to associate to each device, and in which order the devices will be represented on-screen.
- Specify which time marker buttons will be present on-screen, with which colour and at what position.
- Specify, if desired, which metronome-like effects are made available.

- Specify the size in pixels of the two application windows that will appear on your screen, tuning them according to your screen real estate and your needs.

The configuration file shown in Fig. 2.1 generates the screen that can be seen in Fig. 1.1 above.

The main block

```
<vibrion data_dir='dir' bt_unit='unit'>
...
</vibrion>
```

The main element of the file will be called **vibrion**. Its attributes are:

- dir** The directory under which the data directories for each run will be created. The directory will be created if it does not exist. In this case, the user who runs the script must have write permission for its parent directory.
- unit** The progressive number of the Bluetooth board to be used - see Sec. 2.1. **0**, for `/dev/hci0`, **1**, for `/dev/hci1`, etc.

The device block

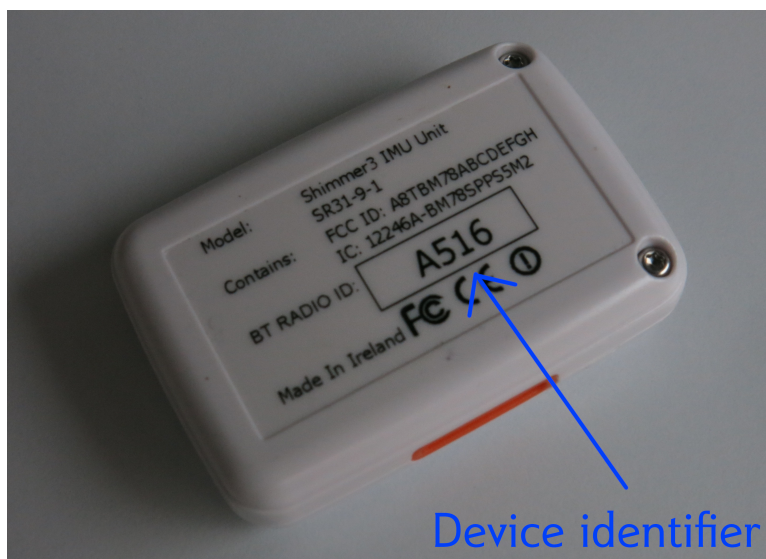


Figure 2.2: Device close-up

```
<dev id='dev_id' tag='tag'>
  <screen_pos x='x_pos' y='y_pos' />
  <graph_pos x='g_x_pos' y='g_y_pos' />
  <data_coll elems='data_collection_fields' />
</dev>
```

One of these blocks has to be included for each Shimmer device that you wish to use in your session. The fields marked in red above must be filled as follows:

- dev_id** The unique identifier of a device. It is a 4-digit hexadecimal value that can be read at the back of each device (Fig. 2.2 shows you where this identifier can be found).

tag A descriptive tag that you associate with this device. It is free-form, but should be kept short, and, hopefully, space-free. With this tag you will be able to identify each device with its prospective use (e.g. on which limb of the body it will be tied).

x_pos, y_pos The location on the display screen real estate where the device's data will appear. The software will fail if two devices have been assigned the same coordinates.

g_x_pos, g_y_pos The location on the graph screen real estate where the device's data will appear. The software will fail if two devices have been assigned the same coordinates. *This field is facultative. If not present, no live graph will be displayed for this device.*

data_collection_fields A comma-separated list of one or more of the fields listed in Table 2.2

Field	Description
A_ACCEL	Request data from the low-noise accelerometer
BMP180_PRESSURE	Request temperature and barometric pressure data
BRIDGE_AMP	So-called "bridge amplifier" (no available data)
D_ACCEL	Request data from the wide-range accelerometer
EXG1_16BIT	One of four fields that appear to be related to a so-called ExG expansion board.
EXG2_16BIT	See above
EXG1_24BIT	See above
EXG2_24BIT	See above
EXT_A15	One of three fields that appear to be related to onboard analog-to-digital converters
EXT_A5	See above
EXT_A6	See above
GSR	Apparently related to a so-called GSR+ expansion board.
INT_A1	One of four fields that appear to be related to onboard analog-to-digital converters
INT_A12	See above
INT_A13	See above
INT_A14	See above
LSM303DLHC_MAG	Requests data from the magnetometer
MPU9150_GYRO	Requests data from the gyroscope
VBATT	Requests data relating to the currently battery charge level

Table 2.2: Data collection fields

Table 2.2 contains all valid fields, as obtained by reverse-engineering of the code. **v.i.b.r.i.o.n** is tested for use only with fields A_ACCEL, D_ACCEL, LSM303DLHC_MAG and MPU9150_GYRO (with data mapped to results produced by Shimmer) as well as BMP180_PRESSURE and VBATT (data mapped to a 0.0...1.0 range).

The marker block



Figure 2.3: Marker condition

```
<marker x='x_pos' y='y_pos' col='background_colour' tag='tag' type='btn_type'/>
```

One of these blocks has to be included for each marker that is to be present on-screen.

Markers are pushbuttons that appear on the lower part of the screen of the application, that are intended for researchers to keep note of events during experimental sessions.

The only consequence when one of these buttons is pressed is that a record about the event is saved in session data. This data (timestamp and button pressed) will then be dumped to a dedicated file when the data production script is run.

Markers' buttons can be of one of three types:

- **simple**: every time they are clicked, they generate an event, with the name assigned as tag to the marker.
- **one_btn, two_btn**: markers of these types have a condition that is stored within the system. The condition (active / inactive) is indicated in the display in the way shown in Fig. 2.3. The two types differ in how the condition is changed:
 - For markers of type **one_btn**, the condition cycles between active and inactive each time a mouse button is pressed.
 - For markers of type **two_btn**, the condition is set to **active** by pressing the left button of the mouse, and to **inactive** by pressing the right button of the mouse.

The events that are saved in the result file are composed by the tag assigned to the marker followed by a space, and string **start** if the condition enters active state, or **stop** if the condition enters inactive state.

The fields marked in red above must be filled as follows:

x_pos, y_pos The location on the display screen real estate where the device's data will appear. The software will fail if two devices have been assigned the same coordinates.

The horizontal size of each button is defined by the maximum column number being used. Markers will use the same horizontal space as device fields. The higher the maximum column number, the smaller the button field. *The font size used to write the on-screen marker tags is fixed! If you define many columns, it is up to you to define marker tags that are short enough so they do not spill out of the button area.*

background_colour The colour of the background of the specific marker, expressed as a 6-digit hexadecimal number: the first two digits indicate the red intensity, digits three and four indicate the green intensity, and digits five and six indicate the blue intensity (e.g. **ffffff** indicates white, while **0000ff** indicates pure blue).

tag Indicates the short string that is associated with this marker button. This string will be displayed on-screen, *and will also appear verbatim in the results file.*

btn_type Indicates the button type. Possible types, and the string to use for each of them, are listed in the block just above.

Free-form markers

A button labeled **free-form** is present in the marker section. By pressing this button, you are allowed to insert markers with any text.

Markers already inserted will be remembered during the session, and may be quickly reinserted or used as base to generate new marker texts.

A practical example will be shown later in this document.

The metronome block

If desired, **v.i.b.r.i.o.n** may produce upon request cyclical sinewave sound patterns from the on-board audio card. Pre-defined patterns may be created with two formats:

- by specifying the frequency, volume and duration of each note and the duration of each pause, or

- in a simplified format, with just a sound repeating at a given interval, by specifying the frequency of the repetition and the percentage of the period that the note is played, together with the frequency and volume of the actual note being played.

The configuration data is included in an XML element called **metron**:

```
<metron card='card_name' dev='device_number' samplerate='sample-rate'>
  ...
  ...
</metron>
```

The three parameters have to be configured for your specific hardware. App. B includes some suggestions about how to proceed

Inside this XML element you will locate one block for each pattern that you will use in your application. Two block types are allowed, corresponding to the two formats described above:

pattern Describes a sequence of notes and pauses:

```
<pattern name='pattern_name'>
  <bip freq='frequency' len='length' vol='volume' />
  <silence len='length' />
  ...
  ...
</pattern>
```

pattern_name is the unique name of this pattern, that will be displayed on-screen.

frequency is the frequency in Hz of the sinewave being reproduced.

length is the length in seconds of the playback.

volume is the volume of the sound being reproduced. *This parameter controls the amplitude of the sinewave being produced. The audio card volume controls can further limit the result.*

beat Describes a simplified definition of a beat pattern.

```
<beat name='pattern_name' bfreq='beat_frequency' on='on_percentage'
  ↳ afreq='audio_note_frequency' vol='volume' />
```

pattern_name is the unique name of this pattern, that will be displayed on-screen.

beat_frequency is the frequency in Hz at which the sound will be repeated.

on_percentage is the part of the period when the note will be played (from 0 to 1).

audio_note_frequency is the frequency in Hz of the sinewave being reproduced.

volume is the volume of the sound being reproduced.

The windows block

Both the main application window and the graphs window (see Sec. 2.3.5) will appear on the screen with the sizes specified in this block. All other dimensions are computed in proportion of these sizes.

A series of hardcoded factors are used to size the various elements and font sizes. I tuned them to give a pleasant and informative appearance to the two windows for my tests, but you may find that they need tuning for your needs. Unfortunately, this is a long and complex procedure that cannot be described on these pages.

If you need to intervene in this, you might want to produce a screen dump of the window whose appearance you need to modify, and mail it to me along with detailed descriptions of the aspects you need to modify. Maybe I can be of help.

Here is the XML element to describe the window sizes:

```
<windows uiw='main_window_width' uih='main_window_height' gw='graph_window_width'  
↪ gh='graph_window_height' />
```

The fields marked in red above must be filled as follows:

main_window_width, main_window_height Dimension in pixels of the main window, that contains the user interface elements.

graph_window_width, graph_window_height Dimension in pixels of the window that can be opened to display live graphs with incoming data.

2.3.2 Application run

Once you have prepared your XML configuration file, you start the application by executing, from the directory that you obtained during installation (`../RBY-Shimmer2/`), this command:

```
ruby scripts/run.rb path-of-the-configuration-file subject-ID
```

The subject ID must be composed of ONLY uppercase and lowercase letters, digits, and the character “_” (underscore).

The example configuration file shown in Fig. 2.1 can be found in the `docs` directory of the distribution, along with this document. Its name is `ex.xml`: it may be useful as a template for your own files.

To obtain the screen shown in Fig. 1.1, you have to run this command:

```
ruby scripts/run.rb docs/ex.xml 3
```

Data from the run will be collected in directory

```
log-directory/subj_subject-ID.date.time
```

where date and time (formats: `YYMMDD` and `HHMMSS`) are relative to the moment when the session has been started. With the data of the example, this directory might have been created:

```
/tmp/subj_3.240813.093155
```

This directory will contain one file to collect raw data for each requested device, plus one for the raw marker data, and a log file collecting the messages that the program generates during the run. In the current case, for a session that is started and immediately stopped, I obtained this log file:

```
[240813.093155] Audio output bfr size 800 period size 80(0) block size 320 frame size 4  
[240813.093155] ALSAUNIT 0 -> 7  
[240813.093157] bt_close_ll: device 0044/alpha already closed.  
[240813.093157] bt_close_ll: device a516/beta already closed.  
[240813.093157] bt_close_ll: device a511/gamma already closed.  
[240813.093157] bt_close_ll: device a50e/omega already closed.
```

```
[240813.093157] Logthread for </tmp/subj_3.240813.093155> out.  
[240813.093157] Closing metron  
[240813.093157] Closing markers  
[240813.093157] Closing stopw  
[240813.093157] Closing shim_0044_alpha.data  
[240813.093157] Closing shim_a516_beta.data  
[240813.093157] Closing shim_a511_gamma.data  
[240813.093157] Closing shim_a50e_omega.data
```

Refer to this file if you experience something unexpected. When the run is concluded, you are informed about where to find the file. For example:

```
Log file can be found in /tmp/subj_3.240813.093155/log
```

Furthermore, a file called `xml` will contain a copy of the XML configuration file.

The names of files containing raw device data will be in this format:

```
shim_dev_id_tag.data
```

The first device defined in the example configuration file will thus generate a data file with this path:

```
/tmp/subj_3.240813.093155/shim_0044_alpha.data
```

Files are created even if no connection is made with the relative device. They contain *binary data* with raw data messages streamed from the device. See Sec. 2.3.7 about how to obtain data that you can use in your analyses.

Linking with devices

If you obtain the screen shown in Fig. 1.1, this means that you have been able to make contact with your computer's Bluetooth hardware. The next step is to link with one or more Shimmer devices, that have the device IDs mentioned in your configuration file.

The first thing you must do is to power the devices. If they work properly, the leftmost LED of each (blue) should display a fast blink (about 4 times per second).

Then, press button 'Query' on the application interface. The underlying Bluetooth interface will try to link with as many devices as it can.

Fig. 2.4 shows how the screen will appear when two devices (in this case the ones with IDs 0044 and A50E) are found, and are ready to be paired.

Strategy for finding devices

The best possible case is that of having just turned on all devices. In this case, if the devices are in good shape and within the reach of the server device, each of them should be found within a few seconds.

If one or more devices are not found, you have the option, once the message "Querying BT devices" has disappeared, to click the **Query** button a few more times. Hopefully, the missing devices will be detected during these further attempts.

If this has not happened, the best possible strategy is to turn off, and then (after a few seconds) on again *the devices that have not registered*. Then, try again with the **Query** button. **You should not turn off any device that has already been found!**

The strategy of last resort, if you do not manage to connect with all devices, is to leave the program (by clicking

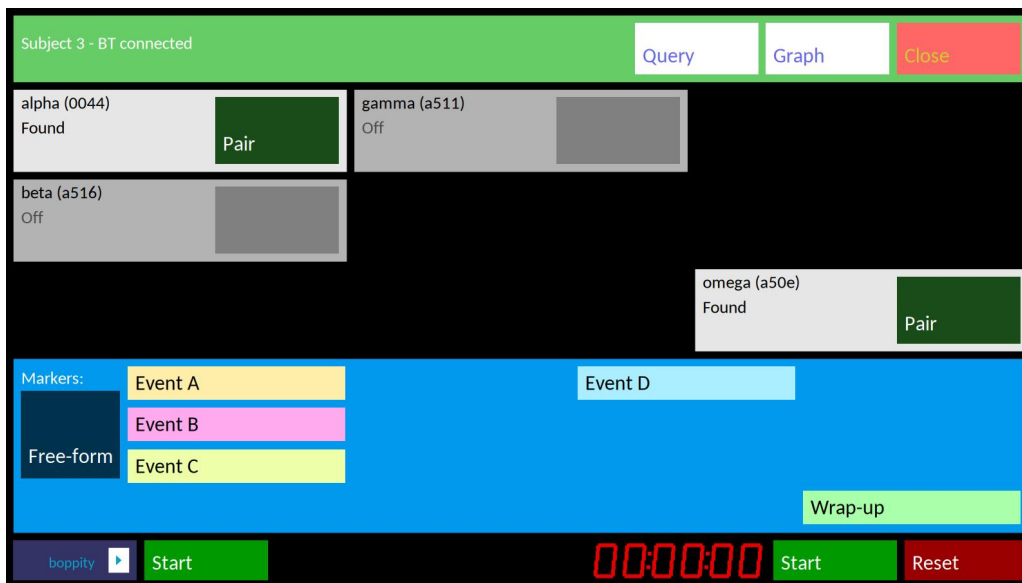


Figure 2.4: Devices were found

the **Close** button), turn off all devices, wait a handful of seconds and restart the whole procedure from the beginning.

Pairing with devices

The on-screen button for all devices that have been found is now labeled **Pair**. When clicking it, the pair process is initiated for that device. It opens a data connection with the specific device, configures it and instructs it to start streaming data.

Only one pairing process at a time can be started. You may click the buttons for every device that is ready to be paired. **v.i.b.r.i.o.n** will start each process in sequence.



Figure 2.5: A device is streaming data

Fig. 2.5 shows how the application screens looks like when a device has been paired and is streaming data.

The blue LED on each devices blinks at a slower pace when streaming is underway.

Signal intensity

When a device is streaming, a 2-digit seven-segment display is present in its area on the main screen. This value represents so-called **RSSI**, or “received signal strength indication” for the specific remote device.

This is a negative value specified in **dB/m**, indicating the signal level of a remote device as reported by the server’s device. The maximum is 0, indicating very close proximity. If you increase the distance of a device to the server, the absolute value will increase, until connection is lost.

Server devices are responsible for reporting this value. You will have to experiment with your device in order to establish the range within which you can operate.

Closing the device stream

While streaming is taking place, the button for the device is labeled **Close**. At any moment, you can press the button. If you do that, streaming is interrupted and the link to the device is closed. The screen output for then device returns to what is shown on Fig. 2.4. From this state, you may click once more on the **Pair** button: the pairing process will be repeated and data streaming will resume.

The result is that data for the period between the closing and the reopening is not collected, but the rest of the data is present and correctly timed.

A record is saved every time data stream is opened and closed. With data production, this information is made available to the researcher in the same file that contains marker data.

*Devices must **not** be turned off while streaming is interrupted. If this is done, you won’t be able to restart streaming.* They can be plugged into the charging base, though, as long as their power switch remains on.

Closing the application

The application is cleanly closed by clicking on the appropriately labeled button at the top-right corner of the screen. Devices that were still streaming at closing time will be properly closed.

2.3.3 Stopwatch

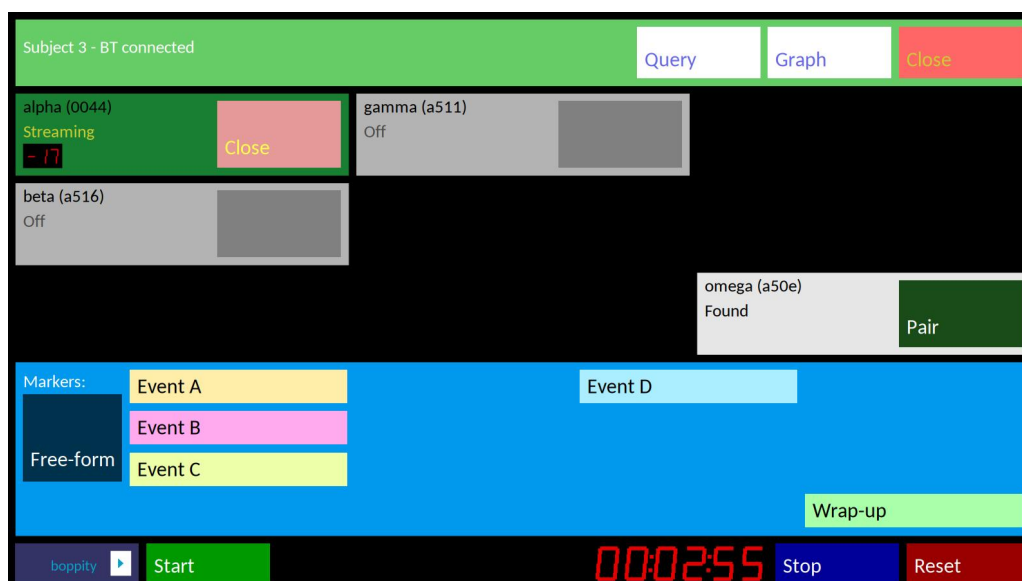


Figure 2.6: The stopwatch is running

The bottom of the screen hosts a second-resolution stopwatch that the experimenter can make use of. Two buttons are available:

- **Start/stop:** The stopwatch is started when not running, or stopped when running.
- **Reset:** The count of the stopwatch is reset to 00:00:00.

Timestamps for interaction with the stopwatch are saved, and eventually included in the marker **csv** file (see Sec. 2.3.7).

2.3.4 Metronome



Figure 2.7: The metronome is running

On the lower-left part of the screen there is a multiple-choice field where the list of metronome patterns that were eventually defined in the XML file can be perused (by clicking on the small arrows). If the **Start** button is pressed, the selected pattern will start being played back through the selected audio device (see Fig. 2.7). While running, the button will be labeled **Stop**, and by pressing it playback will stop.

Timestamps for interaction with the metronome are saved, and eventually included in the marker **csv** file (see Sec. 2.3.7).

2.3.5 Graphs

By clicking on the button labeled **Graph** at the top of the screen, a second window is opened. Click again on that button, and the window is closed. See Fig. 2.8.

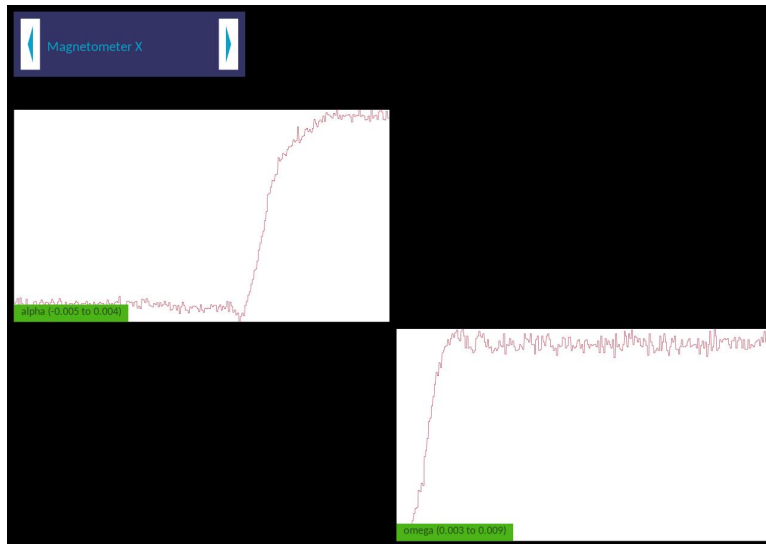


Figure 2.8: The graph window

When configuring your **v.i.b.r.i.o.n** run, in the XML file you can specify, for each sensor device, **graph_pos** variables. If they are specified, an area in this window will be reserved for displaying real-time live data in a graphical form when it is received.

On the top part of the window is an area with clickable left and right-arrow icons. It allows you to select the variable to monitor. In the list are present all variables that are returned by at least one live device.

Every time a new variable is selected, a fresh data collection is started for each device. A line representing freshly collected data starts to be drawn. It is scrolled as needed, with passing time. *The displayed material is scaled LIVE to the current value range.* The minimum and maximum values of the present range are printed on-screen, for each device.

This window is only intended as an aid for verifying that devices are alive and are correctly capturing material.

The material that is plotted is the *raw data* received - i.e. it is not scaled or otherwise modified to match the data produced by the Shimmer software.

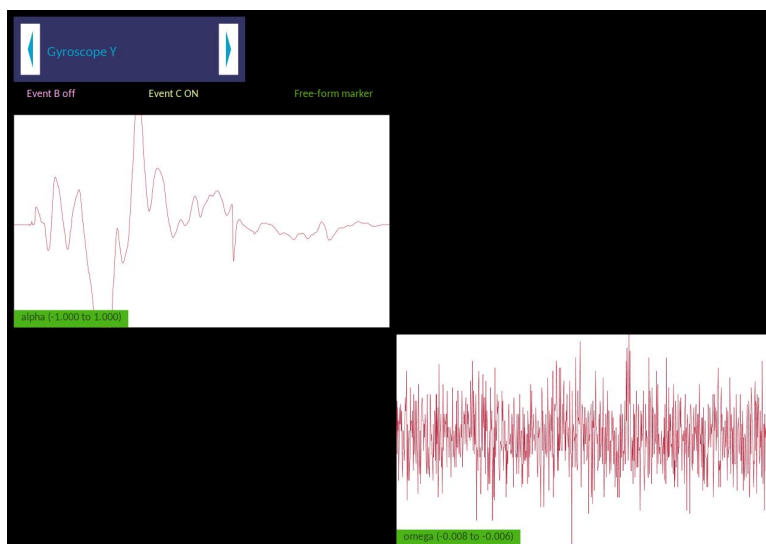


Figure 2.9: Marker information

In the same window, an area between the control box on top and the per-device graphs at the bottom is reserved for the display of eventual markers that have been recently activated.

For each marker that is activated, its tag is displayed in this area, scrolling along time from right to left.

2.3.6 Free-form markers

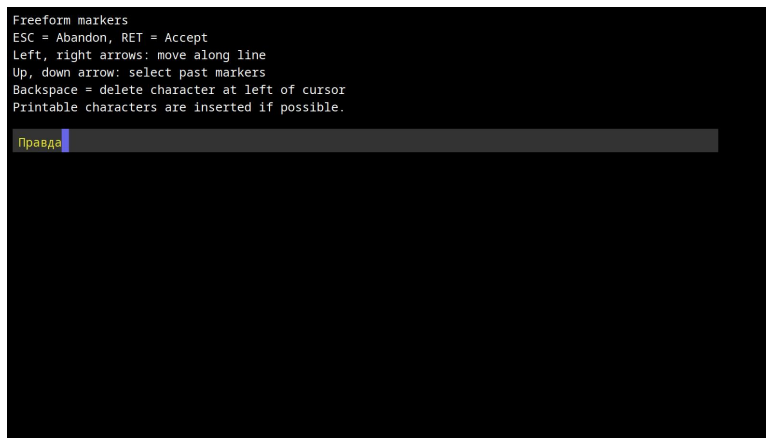


Figure 2.10: Composing free-form markers

Free-form markers are the mechanism that has been put in place to permit the association to specific moments of experiment runs of messages that could not be foreseen in advance.

Messages are composed with the computer keyboard, and when the **Return** key is pressed whatever was composed is sent, and will follow the same processing paths of other marker messages. If the **Escape** key is hit, the message-composing screen is abandoned, discarding any eventual input.

Previously sent messages are kept in memory during a session: you can use the up-arrow and down-arrow keys to scroll along the list in order to either quickly re-send a message or use a previous message as a base for new ones.

An effort has been made to permit the insertion of **UTF-8** characters generated by international keyboard layouts. In Fig. 2.10, for example, you see a message in Cyrillic characters.

The screen includes simple instructions. If you open it by mistake, just press **Escape**, and you will return to the main user interface.

2.3.7 Data production

Once a session is completed, the relative directory will include all relative raw data. These directories can be stored, backed up, compacted, shared... In each of them is contained all data related to a **v.i.b.r.i.o.n** session.

From raw data, data can be produced in a format (**csv**, comma-separated values) that is suitable for importing into spreadsheet programs, or processing in other ways.

Two **csv** files are generated by a post-production script I wrote. You execute it with this command:

```
ruby scripts/shimmer_csv.rb path-of-the-data-area
```

For the example above:

```
ruby scripts/shimmer_csv.rb /tmp/subj_3.240813.093155
```

At the completion of this script, two new files in the **comma-separated** format will exist in the data area of the session:

- `data.csv`
- `marker.csv`

`data.csv` contains all the data that has been collected from the devices in time order (number of seconds from session start). Headers of this table are:

- Time (seconds)
- The identifier of the device (in the format `device-id-device-tag`).
- One column for each data element whose connection was requested for at least one device, *in alphabetical order*.

The second file, `marker.csv`, contains a record for each time an on-screen marker button has been pressed, with its time tag, in the same time coordinates as the data file.

It also includes a time-stamped record for each time streaming for a device has been started and stopped, for each stopwatch action (start, stop, reset), and for every time a metronome pattern has been started or stopped.

TIME	MARKER	START STREAM	END STREAM	STOPWATCH	METRONOME
1.88216					boppity
7.519067					OFF
7.927129					bup
10.471092					OFF
12.255093				Start	
14.221564	Event A start				
14.733081	Event B start				
23.690463		0044_alpha			
24.010453		a50e_omega			
77.647174	Правда				
83.709454	Event A stop				
90.152764				Stop	
92.047177			0044_alpha		
97.351109			a50e_omega		
97.666473		0044_alpha			
103.206466		a50e_omega			
115.159173			a50e_omega		
116.167135			0044_alpha		

Figure 2.11: Spread-sheet detail of a marker file

Fig. 2.11 shows how a simple marker file appears when imported into a spreadsheet. Note that markers are identified by their complete tags as mentioned in the XML configuration file, followed, if needed, by the “start” or “stop” string.

Devices are identified, as usual, by both their device ID and the chosen tag. Metronome patterns are also identified by the name specified in the XML file.

Appendices

Appendix A

Configuration variables

By reverse-engineering the *c-sharp* software freely distributed by Shimmer, I have been able to identify a number of configuration messages that the code is configured to send to the devices. I then proceeded to obtain from a device programmed according to desired specifications the values corresponding to those messages, and I hardcoded this set of values into my code. The relative block appears in source file `ext/shimmer/lib/shimmer2_rb.rb` and can be seen in Listing A.1.

```
1 EXG_REGS=[0x00,0x80,0x10,0x00,0x00,0x00,0x00,0x00,0x02,0x01].pack('C*')
CONFIGURATION=[
  [Shimmer_bt::C_SET_ACCEL_RANGE_COMMAND,[0x00].pack('C*')],
  [Shimmer_bt::C_SET_GSR_RANGE_COMMAND,[0x04].pack('C*')],
  [Shimmer_bt::C_SET_MPU9150_GYRO_RANGE_COMMAND,[0x01].pack('C*')],
6  [Shimmer_bt::C_SET_SAMPLING_RATE_COMMAND,[0x51,0x00].pack('C*')],
  [Shimmer_bt::C_SET_INTERNAL_EXP_POWER_ENABLE_COMMAND,[0x00].pack('C*')],
  [Shimmer_bt::C_SET_EXG_REGS_COMMAND,[0x00,0x00,EXG_REGS.length].pack('C*')+EXG_REGS],
  [Shimmer_bt::C_SET_EXG_REGS_COMMAND,[0x01,0x00,EXG_REGS.length].pack('C*')+EXG_REGS],
  [Shimmer_bt::C_SET_SENSORS_COMMAND,nil],
11 [Shimmer_bt::C_SET_ACCEL_SAMPLING_RATE_COMMAND,[0x07].pack('C*')],
  [Shimmer_bt::C_SET_MAG_SAMPLING_RATE_COMMAND,[0x03].pack('C*')],
  [Shimmer_bt::C_SET_MPU9150_SAMPLING_RATE_COMMAND,[0x12].pack('C*')], # I hope this is right

  [Shimmer_bt::C_INQUIRY_COMMAND,nil]
16 ]
```

Listing A.1: Device configuration

The last message, `C_INQUIRY_COMMAND`, concludes the configuration phase by requesting from the device a reply message with current information. When this reply message is received, the device is put into streaming mode.

Table A.2 lists each message along with the values that I have adopted, and, if possible, some information about the message.

The most puzzling aspect of the list of configuration values is the multiple number of parameters that appear to concur in setting the sampling rate of the devices. The ideal situation would have been for you to program a sampling rate (say, with `SET_SAMPLING_RATE_COMMAND`), from which the device would compute any other range that it would need.

The switch by the device maker from version 2 to version 3 must have been quite radical. Shimmer adopted chip **MPU9150**, apparently the first available self-contained combination of gyroscope, accelerometer and magnetometer. This chip appeared in 2012, when California-based Invensense put together their older MPU6050 (gyroscope and accelerometer) with a magnetometer produced by Japanese company Asahi Kasei.

The board appears to also host a LSM303DLHC chip, which contains another accelerometer and another magnetometer.

Thus, there appear to be, onboard, not only two accelerometers but also two magnetometers.

Message	Values	Notes
SET_ACCEL_RANGE_COMMAND	00	Presumably, sets the range of the accelerometer (which of the two?)
SET_GSR_RANGE_COMMAND	04	Presumably related to the GSR+ expansion board
SET_MPU9150_GYRO_RANGE_COMMAND	01	Presumably, sets the range of the gyroscope
SET_SAMPLING_RATE_COMMAND	51 00	Sets the sampling rate of the device - i.e. the rate at which samples are generated. The value indicated, equivalent to decimal 81 means that the interval between samples is <i>81 32767ths of a second</i> , which translates to about 404.5 samples per second
SET_INTERNAL_EXP_POWER_ENABLE_COMMAND	00	Presumably turns on or off the power to some kind of onboard external connector
SET_EXG_REGS_COMMAND	Bytes defined in EXG_REGS constant	Two copies are sent. Presumably related to the ExG expansion board.
SET_SENSORS_COMMAND	Depends on the configuration	Defines the subset of data sets to be produced. <i>nil</i> seen in the definition is substituted with the required value later in the code.
SET_ACCEL_SAMPLING_RATE_COMMAND	07	Presumably configures the sampling rate of one or both accelerometers. The relationship with SET_SAMPLING_RATE_COMMAND above is unclear.
SET_MAG_SAMPLING_RATE_COMMAND	03	Presumably configures the sampling rate of the magnetometer. The relationship with SET_SAMPLING_RATE_COMMAND above is unclear
SET_MPU9150_SAMPLING_RATE_COMMAND	12	See box

Table A.2: Configuration values

The firmware within the Shimmer devices has the role of dealing with this hardware salad and return data that my code has to process further. I gave to my code the task to return IMU values that match expected ranges and conventions about reference axes.

This is why this code has had the goal of matching *one* usage framework, returning numbers that correspond to a provided model.

This code matches the values that Shimmer code produces in their CSV files, quirks and all, at the time these notes are written, when employing devices that have been configured in one specific way.

Any deviation from that (new hardware, added functionalities, updated firmware, different setup frames) may well brings you into multivariate chaos territory.

Appendix B

Audio setup

The code that manages generation of sounds for the metronome facility requires direct access to the Linux sound subsystem. This means first of all that, to make use of the metronome facility, *pulseaudio* and *pipewire*, user-level processes that get installed on most Linux distributions and that reserve to themselves exclusive access to the audio hardware, must be removed or at least disabled.

This command should provide for removal on Debian-like distributions:

```
apt-get purge pulseaudio pipewire
```

This may result in the removal of many software packages that you need or wish to maintain. For this reason, purging may be too radical for your use.

It may be possible, and sufficient, to stop the pulseaudio *service*. For that, you are on your own. The important is that, when running `aplay -l`, you will not see any reference to pulseaudio, but rather references to the actual audio hardware installed.

It is also necessary to enable group *audio* for the user that will run the process! As super-user, edit file `/etc/group` so that line

```
audio:x:29:user1,user2
```

will read

```
audio:x:29:user1,user2,your-user
```

```

**** List of PLAYBACK Hardware Devices ****
card 0: NVidia [HDA NVidia], device 3: HDMI 0 [HDMI 0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 0: NVidia [HDA NVidia], device 7: HDMI 1 [HDMI 1]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 0: NVidia [HDA NVidia], device 8: HDMI 2 [HDMI 2]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 0: NVidia [HDA NVidia], device 9: HDMI 3 [HDMI 3]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: sofhdadsp [sof-hda-dsp], device 0: HDA Analog (*) []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: sofhdadsp [sof-hda-dsp], device 3: HDMI1 (*) []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: sofhdadsp [sof-hda-dsp], device 4: HDMI2 (*) []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: sofhdadsp [sof-hda-dsp], device 5: HDMI3 (*) []
  Subdevices: 1/1
  Subdevice #0: subdevice #0

```

Figure B.1: Output of `aplay -l`

Here I will describe a method to correctly detect the audio card and device values that you will need to insert into the **v.i.b.r.i.o.n** configuration file.

- Disable pulseaudio.
- Install software **audacity**.
- Get hold of an audio file (any MP3 song file will do).
- Connect a headphone to the audio output you plan to use.
- Start Audacity:

```
audacity audio-file
```

- From the **Audio setup** menu, choose **Playback device**. A list of possible choices will appear. Try to play your file with each choice, until you hear its playback from your headphones.

Before you do this, make sure, by making use of a suitable audio mixer program (e.g. **alsamixer**), that levels associated with the various playback cursors are not at 0. Once you locate the right board/device combination, you can later go back to the mixer control to make sure that playback is at a comfortable level.

Fig. B.2 shows the combination that, in my case, produced the required output. Mark down the content of the selected item - in my case:

```
sof-hda-dsp - (hw:1,0)
```

Then run this command:

```
aplay -l
```

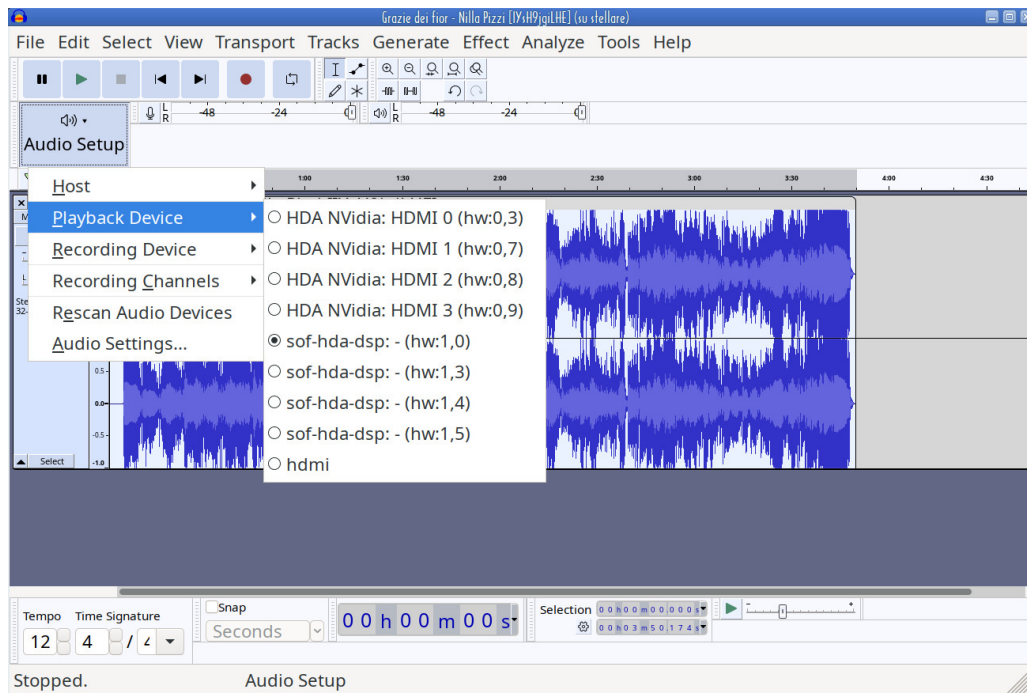


Figure B.2: Running **audacity**

You will receive an output similar to what is shown in Fig. B.1. Locate the first string you marked (in my case, **sof-hda-dsp** together with a device number equal to the number at the right of the comma of the string you had marked down (in my case, 0). You will hopefully isolate just one block. In my case:

```
card 1: sofhdadsp [sof-hda-dsp], device 0: HDA Analog (*) []
Subdevices: 1/1
Subdevice #0: subdevice #0
```

You will then be able to fill the header of the **metron** XML header. Just use the first string in the block (in my case, **sofhdadsp** and the device number:

```
<metron card='sofhdadsp' dev='0' samplerate='8000'>
```

The sample rate figure indicates how many samples per second you will be playing to your audio card. A value of 8000, which is normally used to record and playback telephone-quality voice conversations, is more than enough for a playback of simple sinewaves.